

ETIS

Backup, Recovery & Disaster Recovery Runbook

Controlled recovery of application, database, configuration, identity dependencies, and durable engineering records

v0.18.0 manual edition. Software release candidate; publication follows deployment acceptance.

Audience	ETIS production operators, maintainers, course owners, and designated recovery personnel
Purpose	Provide a safe, testable recovery path that preserves data integrity, provenance, authorization boundaries, and the ability to prove the recovered system is trustworthy.
Production service	https://simulator.etisframework.org
Reference implementation	COMP 330/474 Software Engineering • Fall 2026
Version	v0.18.0 • October 2, 2026

OPERATING PRINCIPLE

Production changes must be deliberate, reversible where possible, observable, and validated. The repository and protected deployment workflow remain authoritative; local tools do not replace controlled deployment.

Contents

Section	Purpose
Executive Overview	Recovery philosophy and decision model

Section	Purpose
1-3	Recovery assets, responsibilities, and pre-incident preparation

Section	Purpose
4-8	Application rollback, PostgreSQL PITR, schema/data validation, secrets/config recovery

Section	Purpose
9-11	External dependency/DNS recovery, disaster scenarios, return to service

Section	Purpose
12	DR exercise procedure

Section	Purpose
Appendices	Recovery checklists, validation queries/concepts, evidence record

Executive Overview

ETIS recovery is not “get the web page back.” A recovered system must preserve the semester lifecycle, memberships, frozen evidence, review sessions/turns, findings/dispositions, AI usage records, and the trust boundaries that make those records meaningful. Recovery therefore treats application images, database state, configuration source, secrets, and external identity/repository dependencies as separate layers.

Recovery decision tree

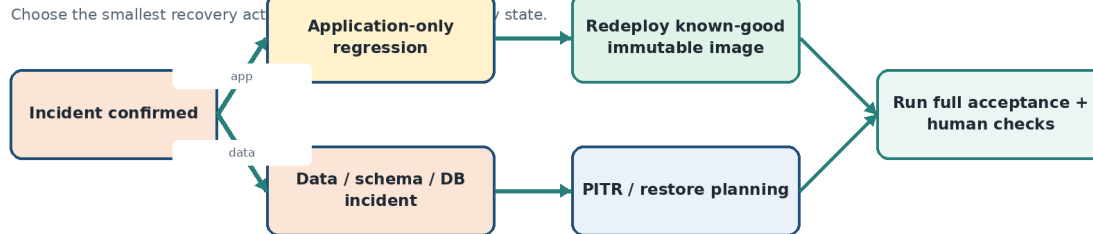


Figure 1. Recovery decision model. Use application rollback for application-only regression; use PITR/restore planning when data or schema integrity is involved. Historical reference figure; current text governs.

RECOVERY PRINCIPLE

Prefer the smallest recovery action that restores a known-good, auditable state. Never overwrite or delete the only known copy of production data while investigating.

1. Recovery layers and authoritative records

Recovery layers and records

Recovery depends on preserving both data and the exact application/configuration state that can interpret it.



Figure 2. ETIS recovery depends on multiple preserved layers: immutable application images, PostgreSQL backups/PITR, durable engineering records, and authoritative configuration source. Historical reference figure; current text governs.

Layer	Authority / recovery asset	Important constraint
Application	Immutable ACR images tagged by Git commit SHA	Do not rebuild an old commit just to recreate an already-retained rollback image.
Database	Azure PostgreSQL Flexible Server platform backup/PITR	Restore to a temporary server first when proving recovery; validate schema and data before cutover.

Layer	Authority / recovery asset	Important constraint
Schema	Alembic migration history/head	Do not manually edit schema to force compatibility.
Infrastructure	Bicep under `infra/azure/` plus protected deployment workflow	Portal state is not the long-term source of truth.
Secrets	Key Vault + protected GitHub environment secret provisioning	Never copy secret values into docs/tickets; rotate if exposure is suspected.
Identity/repository	Entra app/callbacks; GitHub OAuth/App configuration	Authentication and GitHub repository authority remain separate boundaries.
Engineering records	PostgreSQL durable records + frozen evidence provenance	Archive preserves history; recovery must not silently rewrite old evidence/reviews.

2. Recovery objectives and stop conditions

- Primary objective: restore trustworthy course operation without corrupting or rewriting historical engineering records.
- Recovery time is secondary to integrity when the incident involves database/schema uncertainty.
- If the current database may be corrupted, stop normal writes before experimentation.
- If secrets may be exposed, rotate/revoke them as part of recovery; do not merely restore the application.
- If a migration is incompatible with an application rollback, plan database recovery explicitly instead of assuming code rollback is sufficient.

3. Pre-incident readiness

1. Keep known-good Git-SHA application images in ACR.
2. Keep PostgreSQL backup retention/PITR enabled; accepted baseline is 7 days.
3. Maintain the protected `production` GitHub environment and authoritative Bicep.
4. Run post-deployment acceptance after every production change.
5. Periodically perform a restore drill to a temporary private PostgreSQL server and delete it after validation.
6. Ensure operators know where alerting, budget, Key Vault, and database restore controls are located.

The screenshot displays the Azure portal interface for a PostgreSQL Flexible Server. At the top, the server name 'prod-pg-ev3bgd5nmvfg4' is shown with a warning icon and a status 'Are there any outages impacting this resource'. Below this, a toolbar contains actions like 'Connect from VS Code', 'Delete', 'Reset password', 'Restore', 'Restart', 'Upgrade', 'Stop', 'Refresh', and 'CLI / PS'. A yellow banner indicates that the 'Burstable compute tier' is optimized for dev/test workloads. The 'Essentials' section is expanded, showing key details in two columns:

Essentials	
Subscription (move) ETIS Azure Subscription	Endpoint etis-studio-prod-pg-ev3bgd5nmvfg4.postgres.database.azure.com
Subscription ID cc9eecd-2248-4471-b556-ddf230739643	Virtual endpoint Not enabled
Resource group (move) etis-studio-prod	Administrator login etisadmin
Server name etis-studio-prod-pg-ev3bgd5nmvfg4	Configuration Burstable, B1ms, 1 vCores, 2 GiB RAM, 32 GiB storage
Location Central US	PostgreSQL version 16.14
Status Ready	Availability zone 1
Created on 2026-08-19 16:37:48 (UTC)	High availability Not enabled
	Next maintenance ---

At the bottom, there are links for 'Tags (edit)' and 'Add tags'.

Figure 3. PostgreSQL Flexible Server overview. The production database is the durable system of record; Azure exposes a Restore action for point-in-time recovery. Historical reference figure; current text governs.

4. Recovery decision procedure

1. Classify the incident as application-only, database/data/schema, secret/configuration, external identity/GitHub, or infrastructure/network.
2. Freeze or limit writes if data integrity is uncertain.
3. Capture current revision, database/server state, migration status, incident time window, and the last known-good time.
4. Choose the smallest recovery path below.
5. Perform recovery in a way that preserves the original evidence until the new state is validated.
6. Run technical validation, then full ETIS acceptance, then a human user-path spot-check.

5. Application rollback

Application rollback with immutable images



Figure 4. Application rollback redeploys a retained known-good immutable image. It is not a manual traffic switch to an indefinitely retained revision. Historical reference figure; current text governs.

1. Identify the last known-good merged Git SHA and corresponding successful Deploy Azure run.

2. Confirm the immutable ACR image for that SHA still exists.
3. Confirm the target image is compatible with the current database schema. If not, stop and use a coordinated database recovery plan.
4. Redeploy the known-good image through the controlled production deployment mechanism or approved recovery path; preserve the SHA provenance.
5. Verify latest revision and `/ready`.
6. Run `etis-azure acceptance` and affected-path spot checks.

DO NOT REBUILD HISTORY

If the exact immutable image exists, use it. Rebuilding an old commit later can introduce changed base images/dependencies and is not the same artifact.

6. PostgreSQL point-in-time recovery (PITR)

PostgreSQL point-in-time recovery sequence

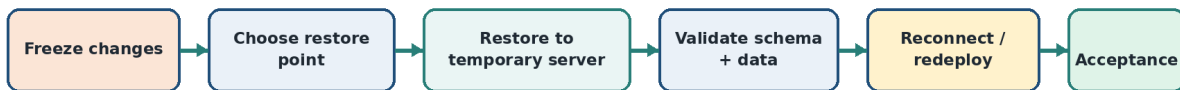


Figure 5. PITR sequence: freeze changes, select a restore point, restore to a temporary server, validate, reconnect/redeploy, and accept. Historical reference figure; current text governs.

6.1 Select the restore point

1. Establish the incident window and last known-good time using application logs, deployment history, and operator reports.
2. Choose a restore timestamp before the damaging event while minimizing legitimate data loss.
3. Record the timestamp and rationale before starting the restore.

6.2 Restore to a temporary server

1. Use Azure PostgreSQL Flexible Server Restore to create a temporary private server at the selected point in time.
2. Keep the original production server intact while validating the restore.
3. Ensure restored networking/DNS/private access allows only the intended validation path.
4. Do not redirect production traffic to the restored server until validation is complete.

6.3 Validate restored database

Validation	What must be true
Connectivity	Authorized validation client/application can connect through the intended private path.
Alembic	Database revision is understood and compatible with the application image selected for recovery.

Validation	What must be true
Core tables	Course term, sections, enrollments, teams, memberships, evidence snapshots, reviews/turns, findings/dispositions, and AI usage records exist as expected.
Incident boundary	Damaging change is absent; expected pre-incident records are present.
Referential integrity	No obvious orphaning or partial recovery in critical authorization/evidence relationships.
Sample user path	A representative authorized user/team can be resolved against the restored data in a controlled validation environment.

Recovery integrity for the current release

Only authoritative corrected or resolved findings are closed. Confirmed, evidence disputed, accepted risk and deferred findings remain open. Inspecting, searching, recommending and completing a review do not close a finding; work after the snapshot cannot close it on that earlier snapshot.

Button-triggered instructions are recorded as review actions, not as words the student wrote. Unsent drafts are preserved and remain private. Optional review context and other concerns stay secondary to the current concern and conversation.

Validate restored snapshot identities/provenance, correction/disposition records and persisted action metadata. Do not recreate later repository fixes as if they existed in an earlier snapshot. Compare an older retained application's transcript behavior with its known capabilities; rollback to an older image may remove later keyboard/authorship improvements.

7. Schema/migration incident

1. If ``ready`` reports `migration_current=False`, first determine whether the migration job failed or the database is at an unexpected revision.
2. Never run ad-hoc DDL to "match" the app.
3. If the migration itself caused data/schema damage, choose a restore point before the migration and pair it with a compatible application image.
4. If a forward fix is safer than rollback, implement it as a reviewed migration on a branch, test it, then deploy through the controlled workflow.
5. Record the pre/post Alembic revisions and acceptance evidence.

8. Secret or Key Vault recovery

1. Determine whether the secret is missing/misconfigured or actually exposed.
2. For exposure: revoke/rotate at the provider first (GitHub App key, OAuth secret, Entra secret, OpenAI key, DB admin credential, session secret as appropriate).
3. Update only the protected provisioning secret path; do not paste values into tickets, chat, screenshots, or documentation.
4. Redeploy/restart only as required for the application to consume the rotated secret.
5. Run ``smoke`` and ``acceptance``; for identity/GitHub secrets, perform a human sign-in/repository verification spot-check.

9. DNS, TLS, Entra callback, or GitHub callback recovery

Symptom	Check	Recovery principle
Custom domain fails	DNS record, Container App custom-domain binding, managed certificate	Restore intended DNS/binding; do not switch users to an undocumented origin.
TLS/certificate error	Managed certificate state and domain validation	Repair certificate/binding; keep HTTPS-only posture.
Entra callback error	ENTRA redirect URI and ETIS_WEB_ORIGIN alignment	Use the canonical https://simulator.etisframework.org/auth/entra/callback .
GitHub OAuth callback error	OAuth App callback and web origin	Restore exact production callback; do not broaden redirect patterns.
GitHub App return flow broken	Setup URL and Redirect on update	Restore /github/setup-complete return path; exact repository verification remains Step 2.

10. Disaster scenarios

Scenario	Primary recovery path	What must be preserved
Bad application deployment	Known-good immutable image redeploy	Current DB unless incompatibility exists; deployment SHA provenance.
Database corruption/erroneous destructive change	PITR to validated temporary server, then controlled cutover	Original server until validation; restore timestamp/rationale.
Failed/incompatible migration	Application/schema compatibility decision; PITR or reviewed forward migration	Alembic revisions, affected data, deployment history.
Secret compromise	Provider revocation/rotation + protected secret update + redeploy/validation	Incident timeline; never preserve compromised secret in docs.
Container App/runtime misconfiguration	Reconcile with Bicep/source and controlled deployment	Source-of-truth; avoid unmanaged portal-only drift.
GitHub App over-scoped	Reduce to selected repositories; revalidate repository trust	Exact authorized repo boundary; do not retain All repositories.
Semester accidentally archived/authority issue	Treat as data/authorization incident; do not edit DB directly	Lifecycle history and membership events; require authorized controlled correction.

11. Return-to-service acceptance

Return-to-service gate

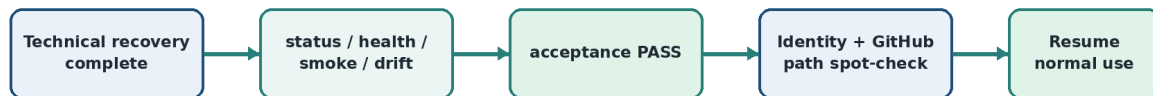


Figure 6. A recovered system returns to service only after technical acceptance and a representative human path check. Historical reference figure; current text governs.

```

./scripts/azure/etis-azure status
./scripts/azure/etis-azure health
./scripts/azure/etis-azure smoke
./scripts/azure/etis-azure drift
./scripts/azure/etis-azure acceptance
  
```

1. Confirm final acceptance is PASS.
2. Confirm restored/redeployed revision and expected min/max replicas.
3. Validate the affected identity/GitHub/review path with a representative authorized account.
4. Confirm archived terms remain read-only and active-term authorization is correct.
5. Confirm no temporary recovery server, widened permission, debug setting, or exposed secret remains.
6. Document the recovery outcome and any accepted data-loss window.

Release-aware recovery acceptance

The v0.18.0 source version is defined in apps/api/app/version.py. FastAPI/OpenAPI and /health use that value. Source version, merged Git SHA, immutable image/revision, deployment success and production acceptance are separate evidence. The August GO records and v0.17 analytical history keep their original dates and versions.

Production defaults remain legacy reasoning and legacy planning. Shadow validator/planner/selector/realizer evaluation remains internal and does not replace student-visible authority. This release introduces no PR5 authority transfer, database migration or course-model change.

Use the version and mode expected for the selected recovery image, not necessarily 0.18.0. Match the immutable image SHA to a compatible Alembic state, then run operator acceptance and affected student/staff smoke. Restoring rows or seeing a page does not by itself prove restored authority or historical integrity.

12. Disaster recovery exercise procedure

1. Schedule the drill outside critical class windows.
2. Record current production revision, database server, Alembic head, and a representative data sample/count.
3. Choose a recent point in time and restore to a temporary private PostgreSQL server.
4. Validate connectivity, Alembic revision, core tables, representative records, and authorization/evidence relationships.
5. Do not switch production traffic during a proof-only drill.
6. Delete the temporary recovery server after evidence is captured and the drill is accepted.
7. Record drill date, operator, restore point, duration, validation results, and lessons/actions.

DRILL SUCCESS CRITERION

A DR drill is not successful merely because Azure created a server. Success means the restored database is connectable through the intended private path, its schema is understood, representative ETIS records are intact, and the recovery process is documented/repeatable.

Appendix A. Emergency recovery checklist

- ☐ Incident time window and current revision captured
- ☐ Write activity limited if data integrity is uncertain
- ☐ Recovery path classified
- ☐ Original database/artifact preserved until validation
- ☐ Restore point or known-good SHA recorded
- ☐ Schema/application compatibility checked
- ☐ Restored/redeployed state validated
- ☐ Full ETIS acceptance PASS
- ☐ Human affected-path check PASS
- ☐ Temporary resources/overrides removed
- ☐ Recovery record completed

Appendix B. Recovery evidence record

Field	Record
Incident ID/date	
Operator	
Failure boundary	
Current/bad revision	
Known-good revision/image	
Database restore point	
Original DB preserved?	
Alembic revision before/after	
Expected/accepted data-loss window	
Acceptance result	
Human path validated	
Temporary resources deleted	
Follow-up actions	

Appendix C. Records that must survive normal archive/recovery

- Course terms/sections and lifecycle state.
- Enrollments and team membership/history.
- Frozen evidence snapshots and provenance.
- Review sessions and persisted turns.
- Findings, corrections, dispositions, disputes, and recommendations.
- Membership events and repository trust state/history as implemented.
- AI usage records required for operational/audit history.

Appendix D. Repository recovery tooling

The repository includes a PostgreSQL backup/restore verification helper used for controlled recovery proof. Run it only in the documented/test context and review the script before pointing it at any environment:

```
./scripts/verify_postgres_backup_restore.sh
```

The existence of the script does not authorize an ad-hoc production restore. Production recovery still requires an explicit restore point, preservation of the original database until validation, schema/data verification, and final production acceptance.